Homework: Summary of Mathematics for Machine Learning

By **Nguyễn Sỹ Hùng** (Student ID: 25MSA33055), 2026-03-08

1. Introduction

This document summarizes the **mathematics topics** introduced in the **Machine Learning 1 course**, based on my understanding and the **lecturer's explanations and guidance**.

For learners or researchers like me, **mathematics provides the precision** and **reasoning** needed to **understand algorithms, evaluate results, detect mistakes,** and **develop new ideas**. Code makes machine learning run, but mathematics makes it understandable.

Machine learning relies on mathematics because **data must be represented numerically**, **predictions must be computed**, **error must be measured**, and **parameters must be updated** to improve performance. The main mathematical areas involved are **linear algebra, statistics, probability, calculus, and optimization**, which together underpin the **machine learning pipeline:** data → model → loss → gradient → update

2. Overview of the mathematics topics

Area	Main question it answers	How it can help
Linear Algebra	How do we represent data and transformations?	It provides the language for vectors, matrices, weights, activations, embeddings, and vectorized computation. It is used in regression, neural networks, PCA, SVD, and recommender systems.
Statistics	How do we summarize and compare data?	It helps describe center, spread, relationships, and variability in data. It is useful for exploratory analysis, feature analysis, error analysis, and model comparison.
Probability	How do we reason under uncertainty?	It gives the foundation for probabilistic prediction, classification confidence, Bayesian reasoning, and uncertain outcomes.
Calculus	How does output change when parameters change?	It explains derivatives, gradients, and backpropagation, which are needed to train models and understand sensitivity.
Optimization	How do we reduce error?	It turns learning into minimizing a cost function. It is used in gradient descent, SGD, Adam, and regularized training.

3. Linear Algebra

3.1. Vectors and Matrices

Concept	Formula	Problem	Explanation
Vector	$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$	Represents one sample or one parameter set	A vector is a point or direction in feature space and is the basic representation unit for inputs, weights, embeddings, and hidden activations.
Matrix	$X \in \mathbb{R}^{m \times n}$	Represents many samples together	A matrix stacks many vectors together, with rows as samples and columns as features. It enables efficient batch and vectorized computation.
Dot product	$x^T y = \sum_{i=1}^{n} x_i y_i$	Measures alignment and builds linear models	It captures weighted combination and alignment between vectors. It is central in linear regression, logistic regression, similarity, and attention-style computations.
Linear model	$\hat{y} = w^T x + b$	Produces prediction from features	This is a weighted sum plus a bias, where each feature contributes according to its weight. It is the foundation of many simple predictive models.

3.2. Norms and Distances

Concept	Formula	Problem	Explanation
L2 norm	$\| x \|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$	Measures vector size	This is the standard geometric length of a vector. It is used in regularization, geometry, similarity, and optimization.
L1 norm	$\| x \|_1 = \sum_{i=1}^{n} \| x_i \|$	Measures total absolute magnitude	
Euclidean distance	$d(x,y) = \| x - y \|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$	Measures closeness between points	It gives straight-line distance and is widely used in clustering, k-nearest neighbors, and similarity search.
Manhattan distance	$d(x,y) = \| x - y \|_1 = \sum_{i=1}^{n} \| x_i - y_i \|$	Measures closeness using coordinate-wise movement	

3.3. Orthogonality and Projection

Concept	Formula	Problem	Explanation
Orthogonality	$x^T y = 0$	Detects independent directions	Orthogonal vectors have no alignment, which makes this idea useful in PCA, basis construction, and decorrelation.
Projection of $x$onto y	$\text{proj}_y(x) = \frac{x^T y}{\| y \|^2} y$	Finds the component of one vector along another	A projection is the part or "shadow" of one vector in the direction of another. It is important in least squares, regression geometry, and dimensionality reduction.

3.4. Eigenvalues and SVD

Concept	Formula	Problem	Explanation
Eigenvalue / Eigenvector	$Av = \lambda v$	Finds special directions preserved by a transformation	The direction stays the same and only the scale changes. This is useful in PCA, covariance analysis, and spectral methods.
SVD	$A = U\Sigma V^T$	Decomposes a matrix into principal directions and strengths	SVD breaks a matrix into rotation and scaling pieces. It is used in PCA, compression, recommender systems, and latent-structure discovery.

4. Statistics

4.1. Describing One Variable

Concept	Formula	Problem	Explanation
Mean	$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$	Finds the center of data	The mean is the average or balance point of the data and is widely used in normalization, summaries, and baselines.
Variance	$\text{Var}(X) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$	Measures spread	It measures average squared distance from the mean and helps detect variability in features and data behavior.
Standard deviation	$\sigma = \sqrt{\text{Var}(X)}$	Measures spread in original units	It is easier to interpret than variance because it is in the same units as the data and is used in z-scores and standardization.

4.2. Relating Two Variables

Concept	Formula	Problem	Explanation
Covariance	$\mathrm{Cov}\,(X,Y) = \mathbb{E}[(X-\mu_X)(Y-\mu_Y)]$	Measures how two variables move together	Positive covariance means the variables tend to move in the same direction, and negative covariance means the opposite. It is used in covariance matrices, PCA, and feature relationship analysis.
Sample covariance	$\mathrm{Cov}\,(X,Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})$	Estimates covariance from observed data	This is the practical version of covariance computed from data and is useful in exploratory analysis.
Correlation	$\rho_{X,Y} = \frac{\mathrm{Cov}\,(X,Y)}{\sigma_X\sigma_Y}$	Standardizes covariance	It gives a unit-free relationship strength in $[-1,1]$, making it easier to compare variables across scales. It is used in feature selection and multicollinearity checks.

4.3. Bias and Variance

Concept	Form	Problem	Explanation
Bias	conceptual	Explains underfitting	High bias means the model is too simple and misses the real pattern. This helps explain poor performance on both training and test data.
Variance	conceptual	Explains overfitting	High variance means the model is too sensitive to the training sample. This helps explain why training performance may be good while test performance is poor.

This tradeoff helps choose model complexity, regularization strength, and training strategy.

5. Probability

5.1. Core Probability Ideas

Concept	Formula	Problem	Explanation

Probability of event	$P(A)$	Measures uncertainty of an event	It gives the chance that an event occurs and supports classification confidence and decision-making.
Joint probability	$P(A, B)$	Measures probability of two events together	It describes overlap between events and is useful in probabilistic models and dependency analysis.
Conditional probability	$P(A \mid B) = \frac{P(A, B)}{P(B)}$	Updates probability with known information	It gives the probability of Aafter learning B, and is central in classification, diagnosis, and Bayesian reasoning.
Marginal probability	$P(A) = \sum_{B} P(A, B)$	Removes dependence on another variable	It gives total probability across hidden possibilities and is used in probabilistic inference.

5.2. Bayes' Theorem

Concept	Formula	Problem	Explanation
Bayes' theorem	$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$	Reverses conditional reasoning	It updates beliefs when new evidence arrives and is used in Naive Bayes and broader Bayesian inference.

5.3. Distributions

Concept	Formula / Form	Problem	Explanation
Bernoulli	$X \sim \text{Bernoulli}(p)$	Models binary outcomes	It describes one yes/no trial and is useful for binary classification targets and logistic regression settings.
Binomial	$X \sim \text{Binomial}(n, p)$	Counts successes across repeated trials	It models repeated Bernoulli outcomes and is useful in count-based problems.
Gaussian / Normal	$X \sim \mathcal{N}(\mu, \sigma^2)$	Models continuous noisy variables	It is the classic bell-shaped distribution and is important in noise modeling, regression assumptions, and standard statistical reasoning.
t-distribution	heavier-tailed than normal	Handles small-sample uncertainty	It is like a more cautious normal distribution and is useful in confidence intervals and hypothesis testing.

5.4. Central Limit Theorem

Statement	Problem	Explanation

Sample means tend toward normality as sample size grows	Explains why many statistical approximations work	Even messy raw data can produce approximately normal averages, which supports confidence intervals and practical inference.

6. Calculus

6.1. Derivatives and Gradient

Concept	Formula	Problem	Explanation
Derivative	$f'(x) = \frac{df}{dx}$	Measures local change	It gives the slope or local sensitivity of output to input and is used in optimization and sensitivity analysis.
Partial derivative	$\frac{\partial f}{\partial x_i}$	Measures effect of one variable while others stay fixed	It isolates the effect of one coordinate in a multivariable function and is crucial in models with many parameters.
Gradient	$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$	Gives direction of steepest increase	It is the multivariable slope vector and directly drives gradient descent and backpropagation.

6.2. Chain Rule

Formula	Problem	Explanation
$\boldsymbol{\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)}$	Differentiates composed functions	It breaks a complex derivative into linked pieces and is essential for backpropagation in neural networks.

6.3. Hessian

Formula	Problem	Explanation
$\boldsymbol{H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}}$	Measures curvature	It describes the shape of the surface, not just the slope, and is useful in second-order optimization and local analysis of minima and maxima.

7. Optimization

7.1. Loss and Cost

Concept	Formula	Problem	Explanation
Loss	$L(y, \hat{y})$	Measures error for one example	It answers "how wrong was this one prediction?" and defines the basic learning signal.
Cost	$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} L(y^{(i)}, \hat{y}^{(i)})$	Measures overall training error	It averages loss across the dataset and becomes the main objective minimized during training.

7.2. Mean Squared Error

Formula	Problem	Explanation
$\mathbf{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Measures regression error	It punishes larger mistakes more heavily because of squaring, and is widely used in linear regression and other regression tasks. It is also smooth and easy to optimize.

7.3. Gradient Descent

Concept	Formula	Problem	Explanation
Gradient descent update	$\theta \leftarrow \theta - \eta \nabla J(\theta)$	Minimizes cost	It updates parameters in the direction that reduces error and is a core training method in regression, logistic regression, and neural networks.
Learning rate	η	Controls update size	It determines step size: too small is slow, too large can overshoot or diverge. It is one of the most important training hyperparameters.

7.4. Regularization

Concept	Formula	Problem	Explanation
L2 regularization	$J_{\text{reg}}(w,b) = J(w,b) + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$	Reduces overfitting	It penalizes large weights and encourages smoother, less extreme solutions, which often improves generalization.
L1 regularization	$J_{\text{reg}}(w,b) = J(w,b) + \frac{\lambda}{m}\sum_{j=1}^{n} \lvert w_j \rvert$	Create sparse models; remove irrelevant features, reduce overfitting	Acts as automatic feature selection, simplifying the model, improving interpretability, and making learning more robust when many features are unnecessary.

8. Logistic Regression

8.1. Model

Concept	Formula	Problem	Explanation
Linear score	$z = w^T x + b$	Builds a raw decision score from features	It combines feature evidence into one raw score before conversion into a probability-like output.
Sigmoid	$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$	Converts score to probability-like output	It squashes any real number into (0, 1), which makes logistic regression suitable for binary classification and probability estimation.

Interpretation: $\hat{y} \approx P(y = 1 \mid x)$

8.2. Cross-Entropy Loss

Formula	Problem	Explanation
$\boldsymbol{L(y, \hat{y}) = -(y \log \hat{y} + (1 - y) \log (1 - \hat{y}))}$	Measures classification error for probability outputs	It penalizes confident wrong predictions heavily and aligns well with probability-based classification, making it more suitable than squared error in this setting.

9. Neural Networks

9.1. Forward Propagation

Concept	Formula	Problem	Explanation
Pre-activation	$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$	Combines previous activations into a raw signal	This is the weighted sum step inside a dense layer and is the core linear transformation in a network.
Activation	$a^{[l]} = g(z^{[l]})$	Introduces nonlinearity	It transforms the raw signal and allows the network to learn more complex patterns than a purely linear model can.

For the first layer, $a^{[0]} = x$.

9.2. Activation Functions

Function	Formula	Problem	Explanation
Sigmoid	$\sigma(z) = \dfrac{1}{1 + e^{-z}}$	Produces probability-like output	It smoothly squashes values into $(0, 1)$and is often used in binary output layers.
Tanh	$\tanh(z) = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Produces centered nonlinearity	It squashes values into $(-1,1)$, which can be useful in hidden layers.
ReLU	$\mathrm{ReLU}(z) = \max(0, z)$	Provides efficient nonlinearity	It passes positive values and blocks negative ones, making it simple and effective for deep neural networks.

Without nonlinear activations, many stacked layers collapse into one linear transformation.

10. Backpropagation

Formula idea	Problem	Explanation

Repeated use of chain rule	Computes gradients efficiently for all parameters	It sends error information backward through the network so each weight can be updated, making neural network training computationally feasible.

11. Core ML training Pipeline – Mathematical View

The entire ML training pipeline can be written almost like a mathematical chain: $D \to x \in \mathbb{R}^n \to \hat{y} = f(x;\theta) \to L(y,\hat{y}) \to J(\theta) \to \theta^* = \arg\min J(\theta) \to$ predictions, which can be interpreted as a flow like this: data → representation → model → loss → optimization → prediction.

Step	Mathematical object	What it does	Explanation
Input	x	Represents features	Encodes raw data numerically so it can be processed mathematically.
Model	$\hat{y} = f(x;\theta)$	Produces prediction	Maps input and parameters to an output.
Loss	$L(y,\hat{y})$	Measures prediction error	Gives the learning signal for one example.
Cost	$J(\theta)$	Aggregates error	Creates one overall objective for optimization.
Gradient	$\nabla_\theta J$	Shows how parameters affect error	Guides how parameters should move to improve performance.
Update	$\theta \leftarrow \theta - \eta \nabla_\theta J$	Improves parameters	Performs the actual learning step iteratively.

12. Conclusion

Mathematics is required for machine learning because it explains each major part of the process. **Linear algebra** represents data and transformations. **Statistics** describes variability and relationships in data. **Probability** handles uncertainty. **Calculus** explains how changes in parameters affect error. **Optimization** provides the method for improving the model through learning. **Together**, these areas show that machine learning is not just code running on data, but a mathematical system for representation, prediction, error measurement, and improvement.

Thank you!

Nguyễn Sỹ Hùng, Mar 2026